Access Control for a Travel Agency Database

I urge you to not copy-paste these questions into search engines or LLMs.

Scenario

You are the DBA for a travel company. Several people need access to the `hotels01`, `flights01`, and `users1` tables, and each should get only what their job requires:

- **ana** is a data analyst. She can read travel data but should never modify it.
- **ben** works the hotel desk. He can adjust hotel nightly prices and nothing else.
- **cara** is a partner from one airline agency. She may see only her own agency's flights.
- **dan** is a senior analyst. He can do everything ana can, and he can also see customer names.

How to test

Run everything as your superuser. To act as another user, use `SET ROLE ana;`. To switch back, use `RESET ROLE;`. `SELECT current_user;` tells you who you are.

> **Tips.** After `SET ROLE`, you have only that role's privileges — superuser powers are dropped, so a "permission denied" error is a real result. `session_user` still shows your original login. In GUI tools such as pgAdmin or DBeaver, each query tab may use its own connection, so keep one tab and check `current_user` often.

Setup

```
Load the sample data provided: travel-schema-small.sql
CREATE ROLE ana;
CREATE ROLE ben;
CREATE ROLE cara;
CREATE ROLE dan;
```

Part A — GRANT

A1. Give ana read access to all three tables. Then, as ana, confirm that `SELECT` works and that this statement fails:

```
DELETE FROM hotels01 WHERE travelcode = 0;
```

A2. Give ben permission to update **only the** `hprice` **column** of `hotels01`, and nothing else. As ben, run:

```
UPDATE hotels01 SET hprice = hprice * 1.10 WHERE travelcode = 0;
```

Is it successfully executed? If not, why does it happen? Fix it with an additional grant you can.

Part B — Authorization on Views

B1. Create a view `agency_flights` that shows only the flights of the agency `'FlyingDrops'`. Grant cara `SELECT` on the view only. As cara, verify that she can query the view but not `flights01`.

B2. Customer names are sensitive. Create a view `users_public` that exposes `usercode, company, gender, age` but not `name`.

1. Grant ana `SELECT` on `users_public`.
2. Revoke her `SELECT` on `users1`.
3. As ana, query `users_public`, then `users1`.

Explain what happens.

Part C — Roles

1. Create a role `analyst` with `SELECT` on `hotels01`, `flights01`, and `users_public`. Grant `analyst` to ana.
2. Create a role `senior_analyst` that **inherits** `analyst` and also has `SELECT` on `users1`. Grant `senior_analyst` to dan.
3. As dan, query `flights01` and `users1`.

Can dan access the tables? Where did each privilege come from?